CORE JAVA NOTES

A practical revision guide from fundamentals to advanced concepts

FOUNDATIONS	OOP & LANGUAGE
Variables, types, operators	Classes, inheritance, polymorphism
Control flow, arrays, strings	Interfaces, abstraction, encapsulation
CORE APIS	ADVANCED TOPICS
Collections, generics, exceptions	Streams, lambdas, concurrency
I/O, dates, regex	JVM, memory, best practices

Includes explanations, syntax examples, comparison tables, and interview revision points.

Contents

• 1. Java overview and program structure

• 2. Variables, data types, operators and control flow

• 3. Arrays and strings

• 4. Object-oriented programming

• 5. Classes, constructors, keywords and access control

• 6. Exception handling

• 7. Collections Framework and generics

• 8. Generics, equals/hashCode and comparable/comparator

• 9. Lambdas, functional interfaces and Stream API

• 10. File I/O, serialization, dates and regular expressions

• 11. Multithreading and concurrency

• 12. JVM, memory management and garbage collection

• 13. Common interview comparisons and best practices

• 14. Practice questions

1. Java Overview

Java is a high-level, class-based programming language designed to be portable. Java source code is compiled into bytecode, which is executed by the Java Virtual Machine (JVM).

JDK vs JRE vs JVM

• **JVM:** Executes Java bytecode and provides runtime services such as memory management.

• **JRE:** The runtime components needed to run Java applications; conceptually includes the JVM and libraries.

• **JDK:** Development kit containing tools such as the compiler (javac), launcher (java), debugger, and runtime components.

Basic program structure

```
public class Main {
    public static void main(String[] args) {
        System.out.println("Hello, Java!");
    }
}
```

main method: the standard application entry point. **static** means it belongs to the class, **void** means it returns no value, and **String[] args** receives command-line arguments.

Compile and run

```
javac Main.java
java Main
```

2. Variables, Data Types, Operators & Control Flow

Primitive data types

Java has eight primitive types: byte, short, int, long, float, double, char, and boolean. Reference types include classes, arrays, interfaces, enums, and records.

Typical sizes: byte 8-bit, short 16-bit, int 32-bit, long 64-bit, float 32-bit, double 64-bit, char 16-bit UTF-16 code unit. boolean has true/false values; its storage size is JVM-dependent.

Variables

```
int age = 25;
long population = 8_000_000_000L;
double price = 99.50;
char grade = 'A';
boolean active = true;
String name = "Mahadev";
```

• **Local variables:** declared in methods/blocks; must be initialized before use.

• **Instance fields:** belong to each object and receive default values.

• **Static fields:** belong to the class and are shared by its instances.

• **Constants:** use final to prevent reassignment.

Operators and flow

Operators include arithmetic (+, -, *, /, %), relational (==, !=, <, >), logical (&&, ||, !), assignment (=, +=), unary (++/--), bitwise (&, |, ^, ~), and ternary (condition ? a : b).

```
if (age >= 18) {
    System.out.println("Adult");
} else {
    System.out.println("Minor");
}

for (int i = 0; i < 5; i++) {
    System.out.println(i);
}

int n = 1;
while (n <= 3) {
    System.out.println(n++);
}
```

Use **break** to exit a loop or switch, and **continue** to skip to the next loop iteration. Modern switch expressions can return a value.

3. Arrays and Strings

Arrays

An array stores a fixed number of values of one type. Indexes begin at 0; accessing an invalid index throws ArrayIndexOutOfBoundsException.

```
int[] nums = {4, 8, 2};
System.out.println(nums[0]); // 4

for (int value : nums) {
    System.out.println(value);
}
int[][] matrix = {{1, 2}, {3, 4}};
```

String

String objects are immutable. Operations that appear to change a String create another String. String literals may be stored in the string pool.

```
String a = "java";
String b = new String("java");
System.out.println(a.equals(b)); // true
System.out.println(a == b);      // usually false here

String result = a.substring(1); // "ava"
String joined = String.join("-", "core", "java");
```

Use **equals()** to compare content; **==** compares primitive values or reference identity. For repeated modifications, use **StringBuilder**; StringBuffer provides synchronized methods.

```
StringBuilder sb = new StringBuilder();
sb.append("Core").append(" Java");
System.out.println(sb.toString());
```

Useful String methods

• length(), charAt(i), substring(begin, end), contains(), indexOf(), startsWith(), endsWith().

• equals(), equalsIgnoreCase(), compareTo(), toLowerCase(), toUpperCase(), trim()/strip().

• split(regex) splits using a regular expression; replace() and replaceAll() have different semantics.

4. Object-Oriented Programming

Four pillars

• **Encapsulation:** keep state private and expose controlled operations through methods.

• **Abstraction:** expose essential behavior while hiding implementation details, often with interfaces or abstract classes.

• **Inheritance:** a class can extend one class and inherit accessible behavior.

• **Polymorphism:** one interface/reference type can represent different concrete implementations; overridden instance methods are dispatched at runtime.

Overloading vs overriding

Overloading: same method name with different parameter lists; resolved at compile time. Return type alone cannot distinguish overloads.

Overriding: subclass supplies a compatible implementation of an inherited instance method; runtime dispatch selects the implementation. Static methods are hidden, not overridden.

Example

```
class Animal {
    void sound() { System.out.println("Some sound"); }
}
class Dog extends Animal {
    @Override
    void sound() { System.out.println("Bark"); }
}
Animal pet = new Dog();
pet.sound(); // Bark
```

Abstract class vs interface

An abstract class can hold instance state, constructors, and abstract/concrete methods; a class extends only one class. An interface defines a contract and can include abstract, default, static, and private methods; a class can implement multiple interfaces.

Composition

Composition means building a class from objects of other classes (has-a relationship). Prefer composition when inheritance would create tight coupling or an inaccurate is-a relationship.

5. Classes, Constructors, Keywords & Access

Constructors

A constructor initializes a new object, has the class name, and has no return type. If no constructor is declared, Java may provide a default no-argument constructor. Constructors can be overloaded but not inherited or overridden.

```
class Employee {
    private final int id;
    private String name;

    Employee(int id, String name) {
        this.id = id;
        this.name = name;
    }
    public int getId() { return id; }
    public String getName() { return name; }
}
```

Important keywords

• **this:** current object; can call another constructor using this(...).

• **super:** superclass members or constructor; a superclass constructor call must occur first in traditional constructor syntax.

• **static:** class-level member; cannot directly use instance members without an object.

• **final:** variable assigned once, method not overridden, or class not extended.

• **abstract:** incomplete class/method intended for subclass implementation.

• **instanceof:** tests whether an object is compatible with a type; pattern matching can bind the matched value.

Access modifiers

public: accessible wherever the containing type is accessible. protected: same package plus subclass access under Java's protected-access rules. package-private (no modifier): same package. private: within the declaring top-level class/nest.

static, final and finally

static is a class member modifier; **final** restricts reassignment/overriding/inheritance; **finally** is an exception-handling block that normally runs whether or not an exception occurs.

6. Exception Handling

An exception is an event that interrupts normal flow. Java's hierarchy begins with Throwable, which includes Error and Exception.

• **Checked exceptions:** generally must be caught or declared, e.g. IOException.

• **Unchecked exceptions:** RuntimeException and its subclasses; e.g. NullPointerException, IllegalArgumentException.

• **Error:** serious JVM/system problems, generally not handled as ordinary application errors.

```
try {
    int value = Integer.parseInt("123");
    System.out.println(value);
} catch (NumberFormatException ex) {
    System.out.println("Invalid number");
} finally {
    System.out.println("Cleanup");
}
```

throw vs throws

throw actually throws an exception object. **throws** declares possible exceptions in a method signature.

```
static void validateAge(int age) {
    if (age < 18) {
        throw new IllegalArgumentException("Must be 18+");
    }
}

static String readFile() throws java.io.IOException {
```

```
    return java.nio.file.Files.readString(
        java.nio.file.Path.of("notes.txt"));
}
```

Prefer try-with-resources for AutoCloseable resources; it closes them automatically.

```
try (var reader = java.nio.file.Files.newBufferedReader(
        java.nio.file.Path.of("notes.txt"))) {
    System.out.println(reader.readLine());
} catch (java.io.IOException ex) {
    ex.printStackTrace();
}
```

7. Collections Framework

The Collections Framework provides interfaces and implementations for groups of objects. Map is part of the framework but does not extend Collection.

Main interfaces

• **List:** ordered, index-based collection; duplicates allowed. Examples: ArrayList, LinkedList.

• **Set:** unique elements. Examples: HashSet, LinkedHashSet, TreeSet.

• **Queue/Deque:** processing order and double-ended operations. Examples: ArrayDeque, PriorityQueue.

• **Map:** key-value mappings with unique keys. Examples: HashMap, LinkedHashMap, TreeMap, Hashtable, ConcurrentHashMap.

Quick comparison

Type	Typical strength	Ordering / notes
ArrayList	Fast indexed access	Insertion order; middle insert/delete may shift elements
LinkedList	Operations at ends; iterator-based edits	Doubly linked; indexed access is slow
HashSet	Fast average membership checks	No guaranteed iteration order
LinkedHashSet	Membership plus predictable iteration	Insertion order
TreeSet	Sorted unique elements	Typically O(log n) operations
HashMap	Fast average key lookup	No guaranteed iteration order; one null key allowed
TreeMap	Sorted keys	Typically O(log n); null-key behavior depends on comparator/natural ordering

Examples

```
List names = new ArrayList<>();
names.add("Asha");
names.add("Ravi");

Set unique = new HashSet<>(List.of(1, 2, 2, 3));
Map scores = new HashMap<>();
scores.put("Asha", 90);
System.out.println(scores.get("Asha"));
```

Use generics (e.g. List) to get compile-time type safety and avoid unnecessary casts. Prefer interface types for variable declarations when practical.

8. Generics, Equality & Ordering

Generics and wildcards

```
static  T first(List values) {
    return values.get(0);
}

// Producer extends, consumer super:
static void copy(List source,
                 List destination) {
    for (Number n : source) destination.add(n);
}
```

PECS means **Producer Extends, Consumer Super**. Generic type arguments are invariant: List is not a subtype of List.

equals() and hashCode()

If two objects are equal according to equals(), they must return the same hashCode(). Unequal objects may share a hash code. Override both consistently when objects are used as HashMap keys or HashSet elements.

Comparable vs Comparator

Comparable defines a type's natural ordering through compareTo(). **Comparator** defines external/custom orderings and supports composition, e.g. Comparator.comparing(Employee::getName).

9. Lambdas, Functional Interfaces & Streams

A functional interface has exactly one abstract method and can be implemented with a lambda. Common interfaces in java.util.function include Predicate, Function, Consumer, and Supplier.

```
Predicate even = n -> n % 2 == 0;
Function length = String::length;
Consumer print = System.out::println;
Supplier random = Math::random;

List nums = List.of(1, 2, 3, 4, 5, 6);
List result = nums.stream()
    .filter(n -> n % 2 == 0)
    .map(n -> n * n)
    .toList();
System.out.println(result); // [4, 16, 36]
```

Stream pipeline

• **Source:** collection, array, generator, or I/O source.

• **Intermediate operations:** filter, map, flatMap, sorted, distinct, limit; generally lazy.

• **Terminal operations:** collect/toList, forEach, reduce, count, anyMatch; trigger evaluation.

• Streams are not data structures and generally should not be reused after a terminal operation.

• Use parallel streams only after considering workload, ordering, shared state, and measurement.

```
Map counts = List.of("a", "bb", "a", "ccc")
    .stream()
    .collect(java.util.stream.Collectors.groupingBy(
        String::length,
        java.util.stream.Collectors.counting()));
```

10. File I/O, Serialization, Dates & Regex

Modern file I/O

```
Path path = Path.of("data.txt");
Files.writeString(path, "Hello Java");
String text = Files.readString(path);
System.out.println(text);
```

Use java.nio.file.Path and Files for common file operations. For large files, prefer buffered or streaming APIs instead of loading the entire file into memory.

Serialization

Java serialization converts an object graph into a byte stream for supported classes. Serializable is a marker interface; serialVersionUID helps version compatibility. Do not deserialize untrusted data.

Date and time

```
LocalDate today = LocalDate.now();
LocalDate nextWeek = today.plusWeeks(1);
LocalDateTime now = LocalDateTime.now();
DateTimeFormatter fmt = DateTimeFormatter.ofPattern("dd-MM-yyyy");
System.out.println(today.format(fmt));
```

Use java.time types such as LocalDate, LocalTime, LocalDateTime, Instant, ZonedDateTime, and DateTimeFormatter for modern date/time work.

Regular expressions

```
boolean valid = "abc123".matches("[a-z]+\\d+");
String[] parts = "red,green,blue".split(",");
```

11. Multithreading & Concurrency

A thread is an independent path of execution. Concurrency concerns coordinating overlapping tasks; parallelism means tasks run at the same time on multiple processing units.

Creating work

```
Runnable task = () -> System.out.println(
    Thread.currentThread().getName());
Thread thread = new Thread(task);
thread.start(); // starts a new thread
// thread.run() would execute as an ordinary method call
```

Important concepts

• **Race condition:** outcome depends on unsafely interleaved access to shared state.

• **synchronized:** mutual exclusion and memory visibility through a monitor.

• **volatile:** visibility and ordering guarantees for a variable; not a substitute for atomic compound operations.

• **AtomicInteger:** provides atomic operations such as incrementAndGet().

• **ExecutorService:** manages task execution and thread pools; shut it down when finished.

• **Callable/Future:** Callable can return a value and throw checked exceptions; Future represents a result or completion.

• **Deadlock:** threads wait indefinitely for locks held by each other; use consistent lock ordering and avoid unnecessary nested locks.

```
ExecutorService pool = Executors.newFixedThreadPool(2);
try {
    Future answer = pool.submit(() -> 6 * 7);
    System.out.println(answer.get());
} finally {
    pool.shutdown();
}
```

In real code, import java.util.concurrent.*. Prefer higher-level concurrency utilities over manually coordinating many raw threads.

12. JVM, Memory & Garbage Collection

• **Heap:** memory area where objects and arrays are allocated.

• **Stack:** each thread has stack frames for method calls and local execution state.

• **Metaspace:** stores class metadata in native memory in modern HotSpot JVMs.

• **Garbage collection:** automatically reclaims heap memory for objects no longer reachable; it does not guarantee immediate cleanup.

• **JIT compiler:** can compile frequently executed bytecode into native machine code.

• **Class loading:** loads, links, and initializes classes as needed.

A memory leak can still occur in a garbage-collected language when code unintentionally retains references to objects it no longer needs. Close external resources such as files and sockets explicitly.

13. Common Interview Comparisons

Concept	Difference
JDK / JRE / JVM	Development tools / runtime components / bytecode execution engine
== / equals()	Primitive value or reference identity / logical object equality
String / StringBuilder	Immutable text / mutable text builder
ArrayList / LinkedList	Fast indexed access / efficient operations at ends and iterator-local changes
HashMap / ConcurrentHashMap	Not thread-safe by itself / designed for concurrent access
throw / throws	Throws an exception now / declares possible exceptions
final / finally / finalize()	Restriction keyword / cleanup block / deprecated finalization mechanism
Overloading / overriding	Different parameter lists / subclass implementation of inherited instance method
Checked / unchecked exception	Compile-time handling requirement / RuntimeException family generally not required to be declared
Process / thread	Independent running program / execution unit within a process

Best practices: use meaningful names; keep fields private; favor immutability where appropriate; program to interfaces; use try-with-resources; override equals/hashCode together; validate inputs; avoid swallowing exceptions; write tests; and measure performance before optimizing.

14. Practice & Revision Questions

1. What is the difference between JDK, JRE, and JVM?
2. Why is String immutable in Java?
3. Explain == versus equals() with an example.
4. Can a static method be overridden? Explain method hiding.
5. What is the difference between an abstract class and an interface?
6. How do checked and unchecked exceptions differ?
7. Why should equals() and hashCode() be consistent?
8. How does HashMap use hashCode() and equals() conceptually?
9. Compare ArrayList, LinkedList, HashSet, and TreeSet.
10. What are generics and why are they useful?
11. Explain lambda expressions and functional interfaces.
12. What is the difference between map() and flatMap() in streams?

13. What is a race condition? How can it be prevented?
14. What does volatile guarantee, and what does it not guarantee?
15. How does garbage collection work at a high level?
16. Why is try-with-resources preferred for files?
17. What are Comparable and Comparator?
18. What is the difference between a process and a thread?
19. What is immutability, and when is it useful?
20. Write a program to count character frequencies in a String.

Quick Study Plan

• **Day 1:** syntax, data types, operators, conditions, loops, arrays and strings.

• **Day 2:** OOP, constructors, access modifiers, static/final, abstraction and interfaces.

• **Day 3:** exceptions, Collections Framework, generics, equals/hashCode and sorting.

• **Day 4:** lambdas, Stream API, file I/O, dates and regular expressions.

• **Day 5:** threads, concurrency, JVM basics, interview comparisons and coding practice.

These notes are a revision guide, not a substitute for compiling examples and reading the API documentation for the Java version used in your project.